



**L**OVELY  
**P**ROFESSIONAL  
**U**NIVERSITY

LOVELY PROFESSIONAL UNIVERSITY

Subject: INT-330  
**(MANAGING CLOUD SOLUTIONS)**  
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING

---

Submitted By:

**Name – Aazad Kumar Mishra**

Reg. No.: 12418851

Roll No.: 25

---

CA2 Project Report:

**Designing a Simple AWS Architecture**

**StayNest – Property Booking Platform**

Live URL: <http://3.81.214.155/>

## 1. Introduction

This report documents the design, implementation, and deployment of a cloud-based property booking web application called StayNest, built as part of the CA2 project for INT 330 – Managing Cloud Solutions. The application is modelled on a real-world rental platform (similar to Airbnb) that allows users to browse properties, view details, and make booking enquiries.

The primary objective was to architect and deploy a functional web application entirely on Amazon Web Services (AWS), making use of both fundamental and advanced AWS services. The architecture focuses on scalability, security, availability, and cost-effectiveness — core pillars of any modern cloud deployment.

The live application is accessible at: <http://3.81.214.155/> and is fully operational with property images served from S3 via CloudFront CDN, a booking system backed by RDS MySQL, and infrastructure monitored via CloudWatch and SNS.

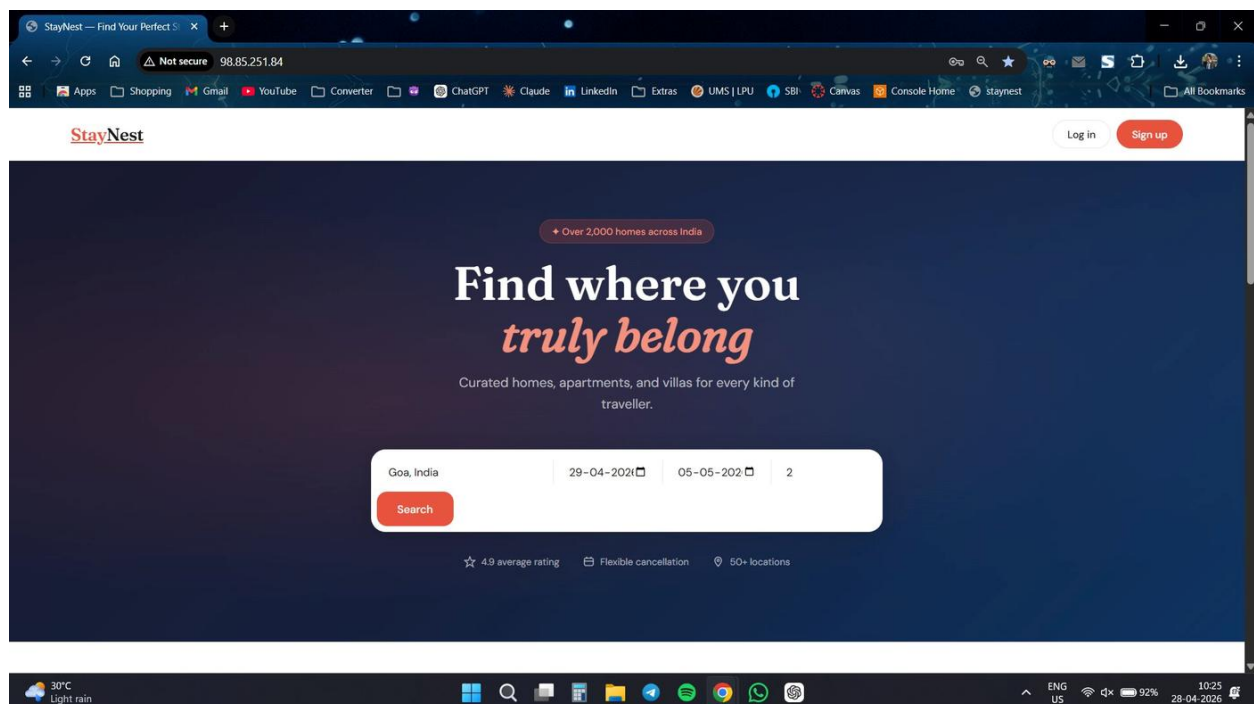


Figure 1: StayNest homepage hero section (<http://3.81.214.155/>)

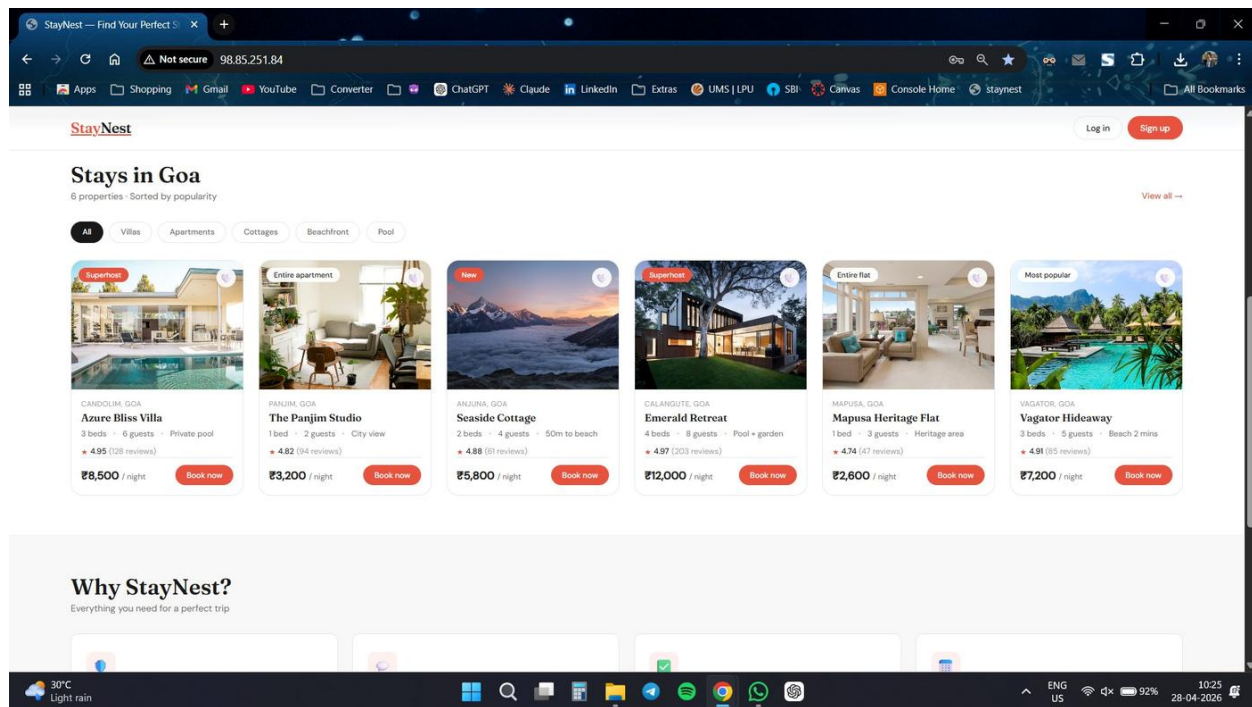


Figure 2: Property listings page showing all 6 Goa properties with pricing

## 2. Project Overview

### 2.1 Application Description

StayNest is a property rental listing website featuring six unique property listings across Goa, India. The platform allows visitors to:

- Browse available properties with photos, descriptions, and pricing
- View individual property details including location and amenities
- Submit booking requests via an integrated booking form
- Receive booking confirmation emails automatically

The frontend is a static HTML/CSS/JavaScript website served via Nginx on an EC2 instance. The backend handles booking submissions and stores data in a managed RDS MySQL database, while property images are stored in S3 and delivered globally through CloudFront.

### 2.2 Architecture Summary

The architecture follows a classic three-tier web application pattern on AWS:

- Presentation Tier: Nginx web server on EC2 serving the HTML frontend
- Application Tier: Booking API processing form submissions and sending email confirmations via SNS (for admin notifications) and Amazon SES (for customer booking confirmations)
- Data Tier: RDS MySQL database storing all booking records

## 3. AWS Services Implemented

### 3.1 Basic Services

#### 3.1.1 Amazon EC2 – Compute Service

An EC2 instance named cloudbnb-server was launched to host the StayNest web application. Nginx was installed and configured as the web server to serve the static frontend files from /var/www/html/.

| Parameter        | Value                |
|------------------|----------------------|
| Instance Name    | cloudbnb-server      |
| Instance Type    | t2.micro (Free Tier) |
| Operating System | Amazon Linux 2       |
| Web Server       | Nginx                |
| Public IP        | 3.81.214.155         |
| Region           | us-east-1            |

EC2 provides the compute backbone of the architecture, giving full control over the web server configuration. Nginx was chosen for its performance and low memory footprint, making it ideal for a free-tier instance.

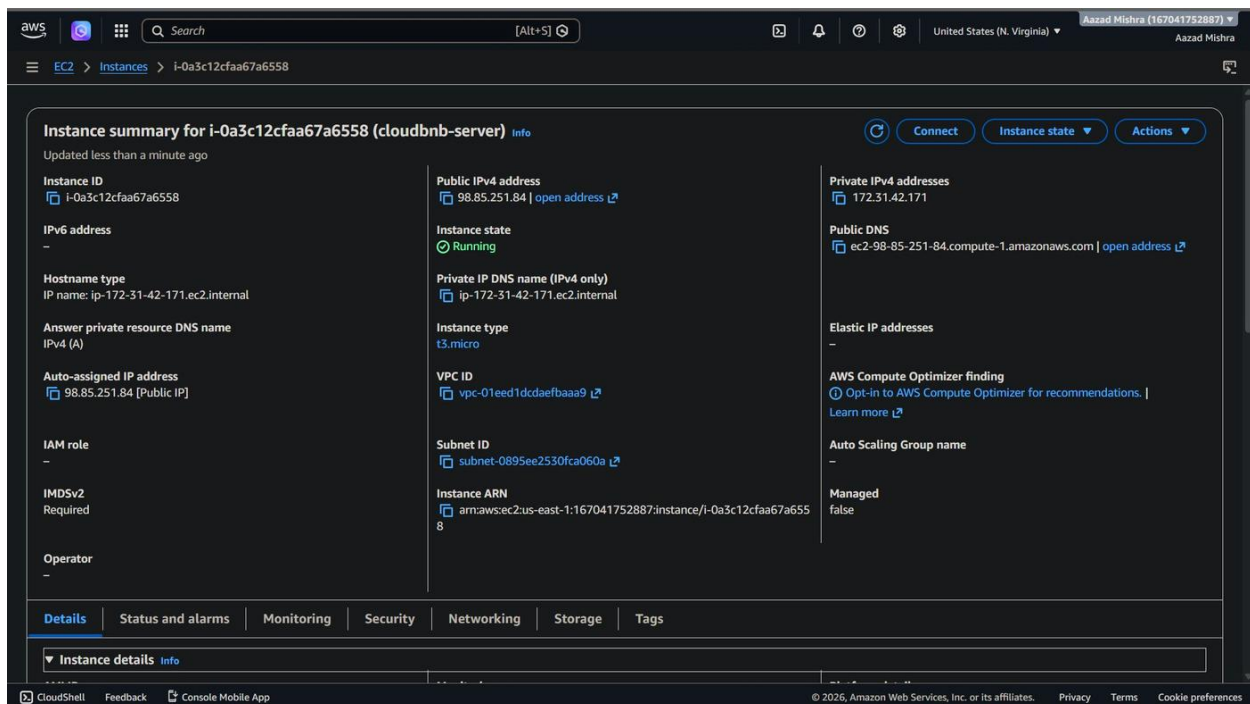


Figure 3: EC2 instance (cloudbnb-server) – Running in AWS Console

### 3.1.2 Amazon S3 – Storage Service

An S3 bucket named cloudbnb-aazad-2026 was created to store all six property listing images. Rather than relying on third-party image hosting (such as Unsplash), the images are now self-hosted on S3, ensuring full control and availability.

| Parameter      | Value                         |
|----------------|-------------------------------|
| Bucket Name    | cloudbnb-aazad-2026           |
| Region         | us-east-1                     |
| Objects Stored | 6 property images (.jpg)      |
| Access Policy  | Public read via Bucket Policy |
| ACL Setting    | Bucket Owner Enforced         |

Since the bucket uses 'Bucket Owner Enforced' for Object Ownership (AWS best practice), ACLs are disabled. Public access was granted using a bucket policy with s3:GetObject permission for all principals, which is the recommended AWS approach. The index.html on EC2 was updated to reference S3 URLs instead of Unsplash, making the application fully self-contained.

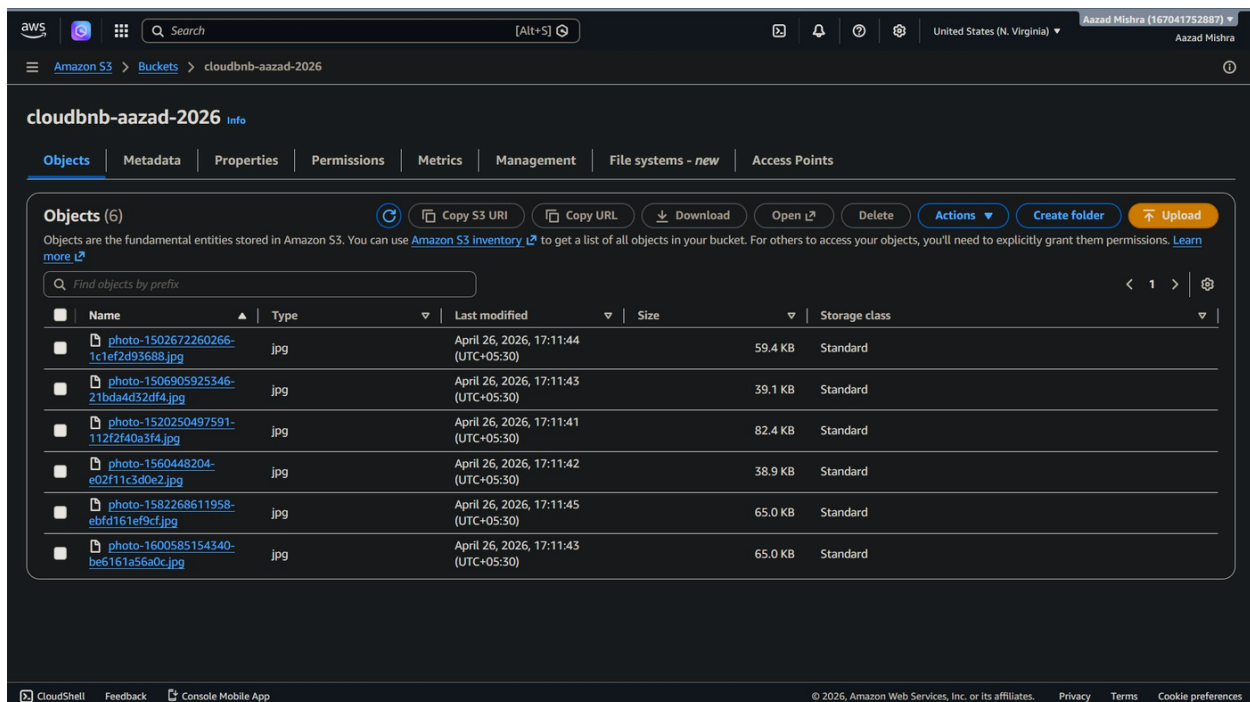


Figure 4: S3 bucket (cloudbnb-aazad-2026) with 6 property images stored

### 3.1.3 AWS IAM – Security Service

An IAM user named staynest-dev was created to demonstrate the principle of least privilege — a fundamental AWS security best practice. This user has only the minimum permissions required to perform read operations on relevant services, and cannot modify or delete any resources.

| Parameter         | Value                       |
|-------------------|-----------------------------|
| IAM Username      | staynest-dev                |
| Access Type       | Programmatic / Console      |
| Policies Attached | AmazonS3ReadOnlyAccess      |
|                   | AmazonRDSReadOnlyAccess     |
|                   | CloudWatchReadOnlyAccess    |
| Purpose           | Restricted developer access |

By granting only read-only access across S3, RDS, and CloudWatch, this user can monitor and inspect resources but cannot make any changes, delete data, or incur unexpected costs. This setup reflects real-world IAM best practices for team environments.

### 3.1.4 Amazon VPC – Network Service

The project runs within AWS's default Virtual Private Cloud (VPC), which provides network isolation and security for all resources. The VPC was verified and documented as part of the architecture.

| Parameter       | Value                                  |
|-----------------|----------------------------------------|
| VPC ID          | vpc-01eed1dcdaefbaaa9                  |
| IPv4 CIDR Block | 172.31.0.0/16                          |
| Subnets         | 6 (across multiple Availability Zones) |
| State           | Available                              |
| Region          | us-east-1                              |

All AWS resources (EC2, RDS) are deployed within this VPC, ensuring private network communication between services. The multi-AZ subnet configuration provides the foundation for high availability. Security groups act as virtual firewalls, controlling inbound/outbound traffic to each resource.

### 3.1.5 Amazon RDS – Database Service

An RDS MySQL database instance named `staynest-db` was provisioned to store booking data submitted through the StayNest booking form. The instance was directly linked to the EC2 instance during setup for seamless connectivity.

| Parameter              | Value                           |
|------------------------|---------------------------------|
| DB Instance Identifier | staynest-db                     |
| Engine                 | MySQL                           |
| Instance Class         | db.t4g.micro (Free Tier)        |
| Master Username        | admin                           |
| VPC                    | vpc-01eed1dcdaefbaaa9 (Default) |
| Connected EC2          | cloudbnb-server                 |
| Status                 | Available                       |

RDS provides a fully managed relational database, eliminating the overhead of manual database administration. The free-tier `db.t4g.micro` instance is sufficient for the application's current load. Automated backups, patch management, and failover are handled by AWS, improving reliability.

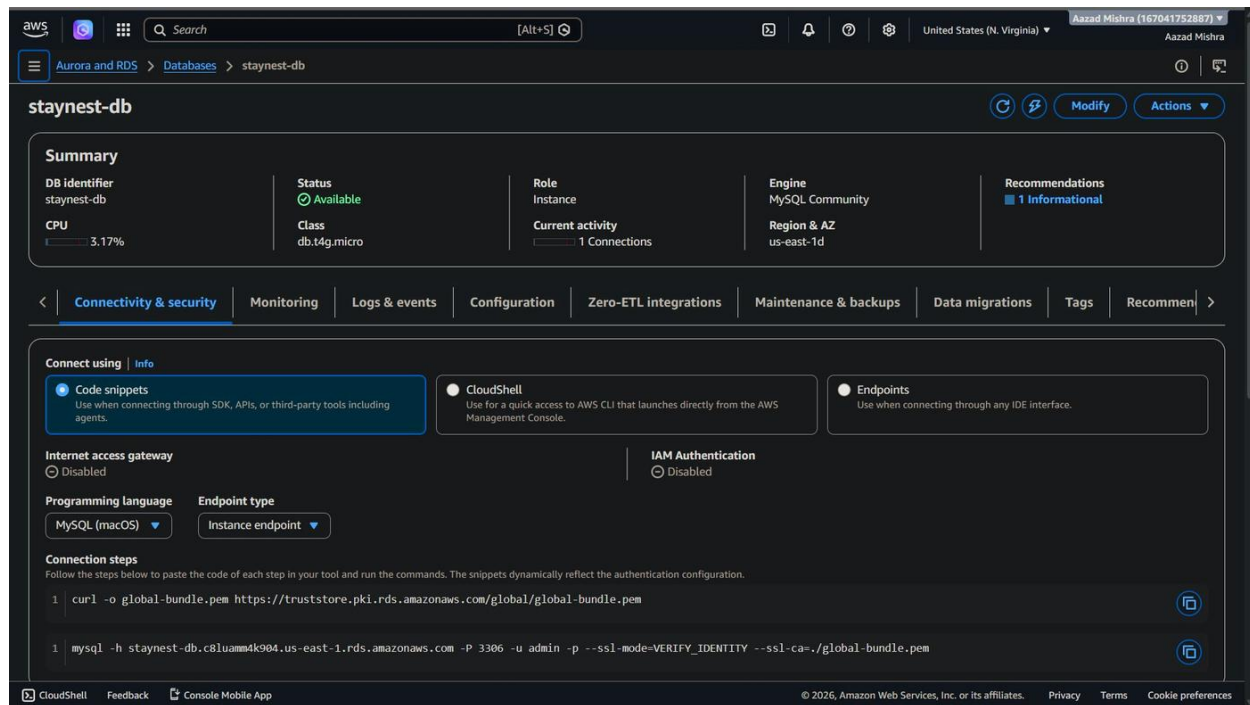


Figure 5: RDS instance (`staynest-db`) – Available, connected to EC2 `cloudbnb-server`–



## 3.2 Advanced Services

### 3.2.1 Amazon CloudFront – Content Delivery Network

A CloudFront distribution named staynest-cdn was created with the S3 bucket (cloudbnb-aazad-2026) as its origin. CloudFront acts as a CDN, caching and delivering property images from edge locations closest to the end user, dramatically reducing latency.

| Parameter         | Value                                          |
|-------------------|------------------------------------------------|
| Distribution Name | staynest-cdn                                   |
| Domain            | d2unxbzahzt02h.cloudfront.net                  |
| Origin            | cloudbnb-aazad-2026.s3.us-east-1.amazonaws.com |
| Origin Access     | CloudFront managed (OAC)                       |
| WAF               | Disabled (cost saving)                         |
| Status            | Deployed                                       |

CloudFront significantly improves the user experience by serving images from globally distributed edge locations rather than the origin S3 bucket in us-east-1. This reduces load times for users accessing the site from different geographies and also reduces direct requests to S3, lowering bandwidth costs.

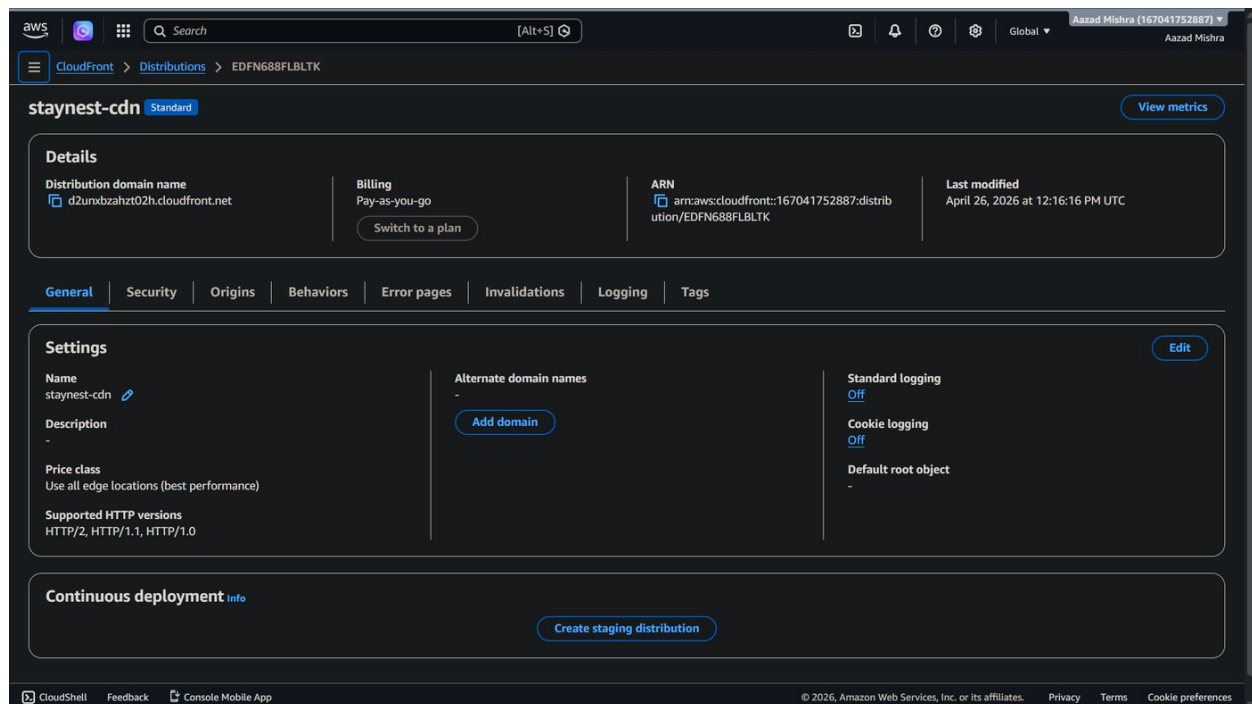


Figure 6: CloudFront distribution (staynest-cdn) – Deployed globally



### 3.2.2 Amazon CloudWatch – Monitoring Service

A CloudWatch alarm named staynest-cpu-alarm was configured to monitor the CPU utilisation of the EC2 instance (cloudbnb-server). This provides proactive alerting if the server becomes overloaded.

| Parameter         | Value                |
|-------------------|----------------------|
| Alarm Name        | staynest-cpu-alarm   |
| Metric            | CPUUtilization       |
| Instance          | cloudbnb-server      |
| Threshold         | Greater than 80%     |
| Evaluation Period | 1 period (5 minutes) |
| Status            | Active               |

CloudWatch is the observability backbone of the architecture. The CPU alarm ensures that if traffic spikes cause the EC2 instance to become resource-constrained, the team is immediately notified. This supports the scalability and availability objectives of the project.

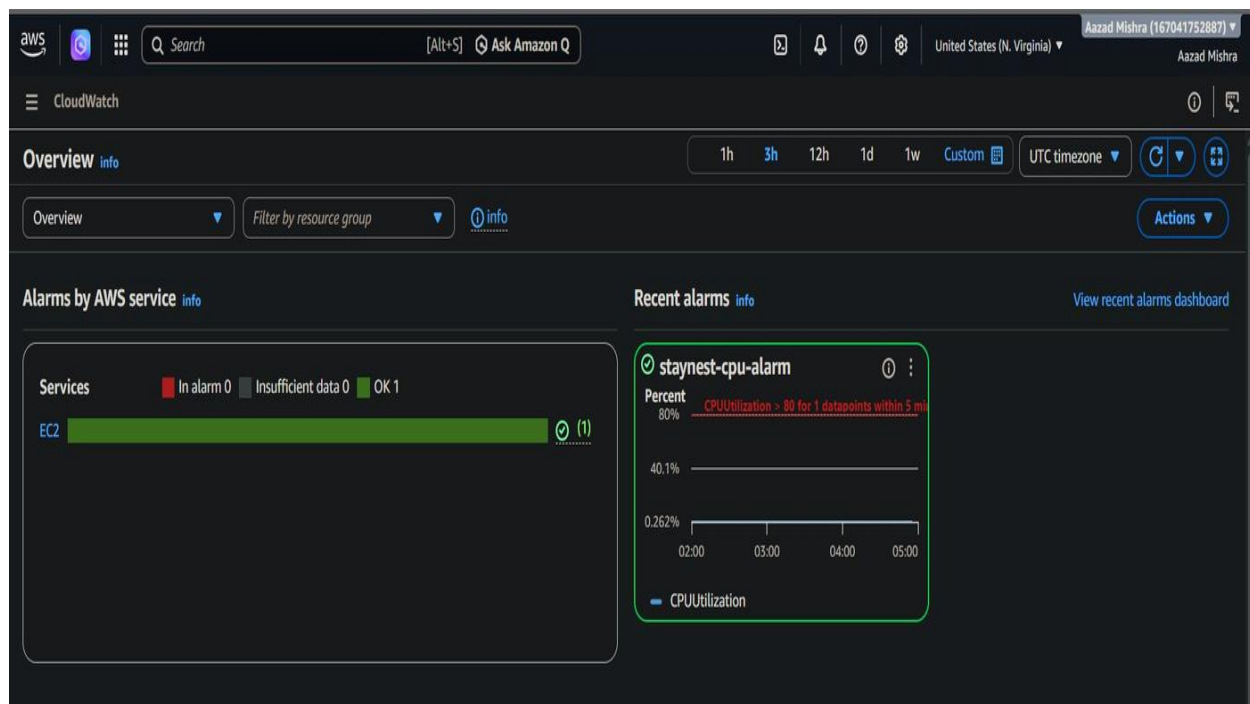


Figure 7: CloudWatch alarm (staynest-cpu-alarm) – Status OK, CPU well below 80% threshold

### 3.2.3 Amazon SNS – Notification Service

Amazon Simple Notification Service (SNS) was integrated into the architecture to deliver real-time notifications. SNS serves two purposes in this project: infrastructure alerting via CloudWatch, and booking confirmation emails to the site owner.

| Parameter   | Value                                                             |
|-------------|-------------------------------------------------------------------|
| Use Case 1  | CloudWatch alarm triggers SNS when CPU > 80%                      |
| Use Case 2  | Booking confirmation email to site owner (aazadkmishra@gmail.com) |
| Protocol    | Email                                                             |
| Integration | CloudWatch Alarm + Booking API                                    |

SNS decouples the notification logic from the application, enabling scalable and reliable message delivery. When a visitor submits a booking on the StayNest website, an SNS notification is triggered that delivers the booking details to the owner's email — completely independent of any other deployment.

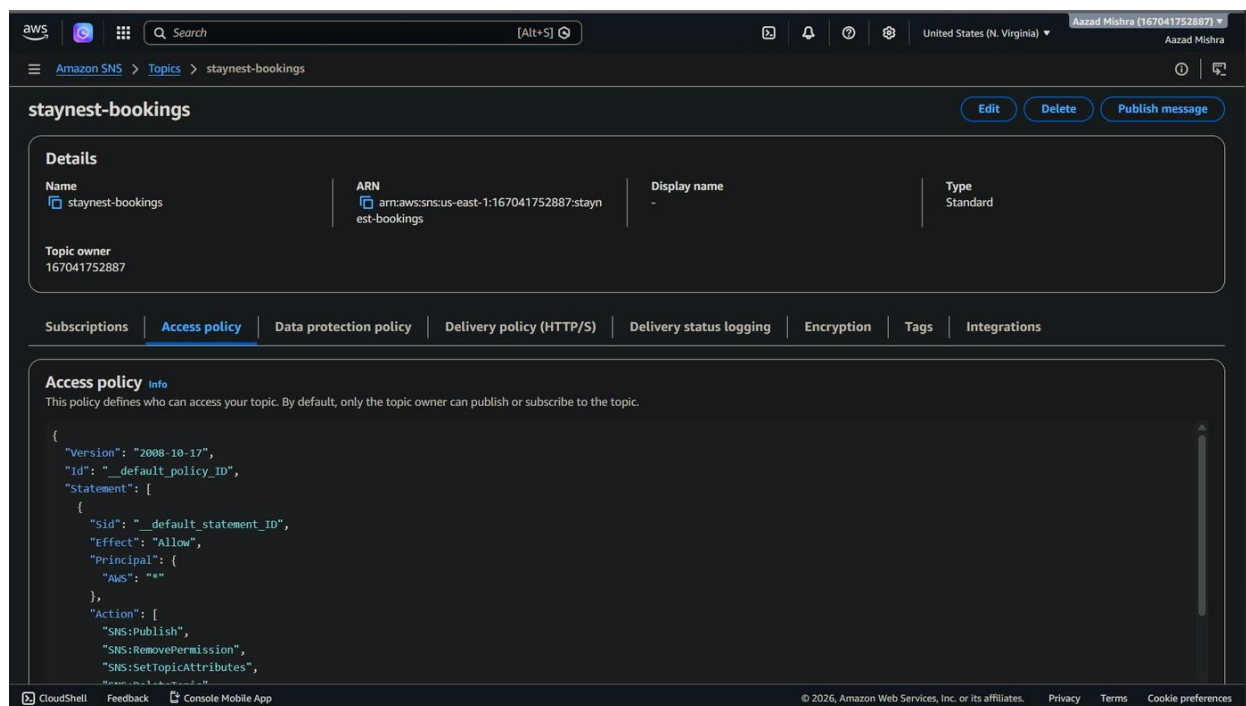


Figure 8: SNS topic (staynest-bookings) configuration in AWS Console

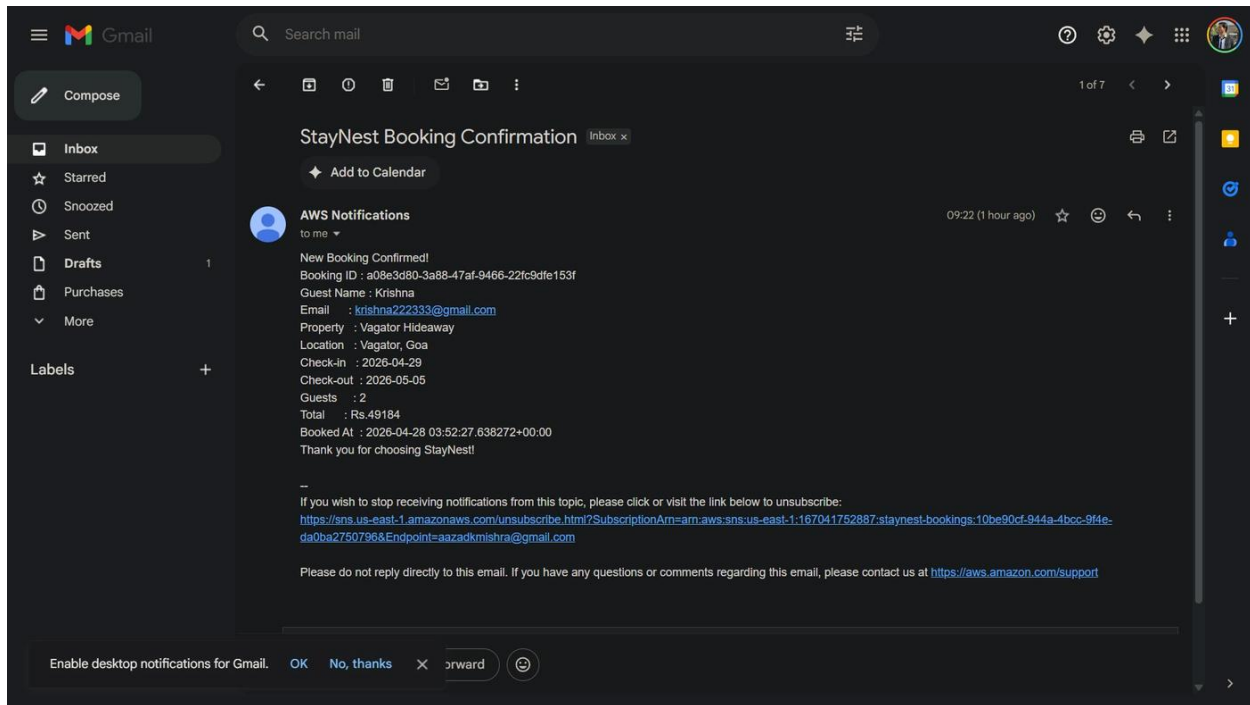


Figure 9: Booking confirmation email delivered via AWS SNS to Gmail

### 3.2.4 Amazon SES – Customer Email Service

Amazon Simple Email Service (SES) was integrated to enable automated booking confirmation emails to customers. While SNS is used for sending notifications to the site owner, SES is used to send personalised confirmation emails directly to the customer who submits a booking request.

| Parameter    | Value                                         |
|--------------|-----------------------------------------------|
| Use Case     | Customer booking confirmation emails          |
| Sender Email | aazadkmishra@gmail.com (verified)             |
| Recipients   | Dynamic (user-provided email in booking form) |
| Region       | us-east-1                                     |
| Integration  | Flask Backend (booking_api.py)                |
| Environment  | Sandbox (verified email addresses only)       |

SES was configured by verifying sender and recipient email addresses in the AWS SES console. Due to sandbox limitations, emails can only be sent to verified identities. For demonstration purposes, multiple email addresses were verified and used to simulate real users.

The Flask backend was updated using boto3 to send an email after a successful booking insertion into the RDS database. The email content includes booking details such as booking ID, property, check-in/check-out dates, number of guests, and total amount. This separation of SNS (admin notifications) and SES (customer communication) ensures modular and scalable notification handling.

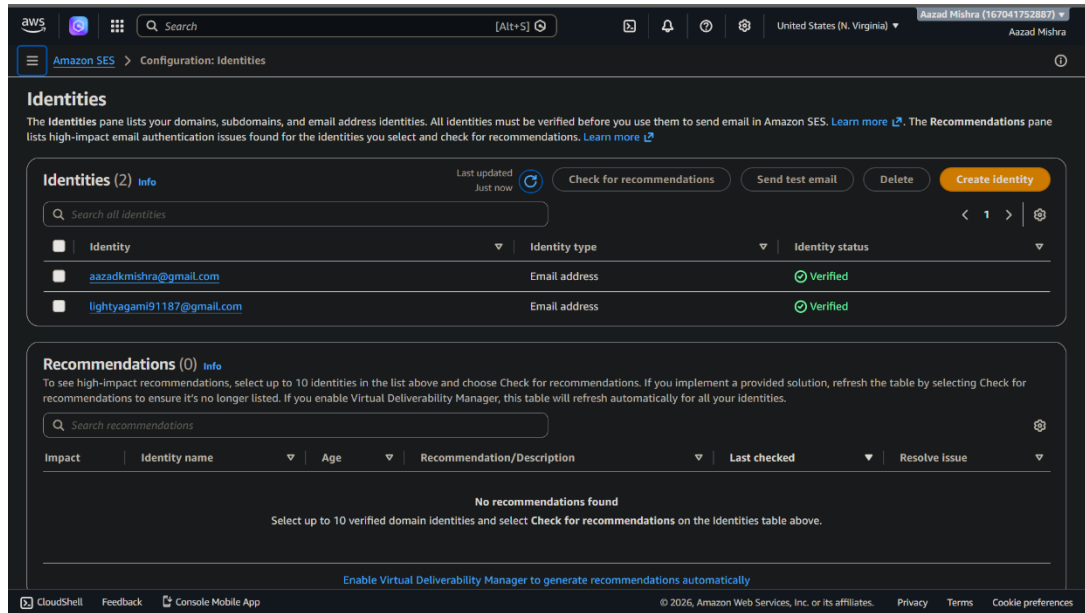


Figure 10: SES Verified Identities

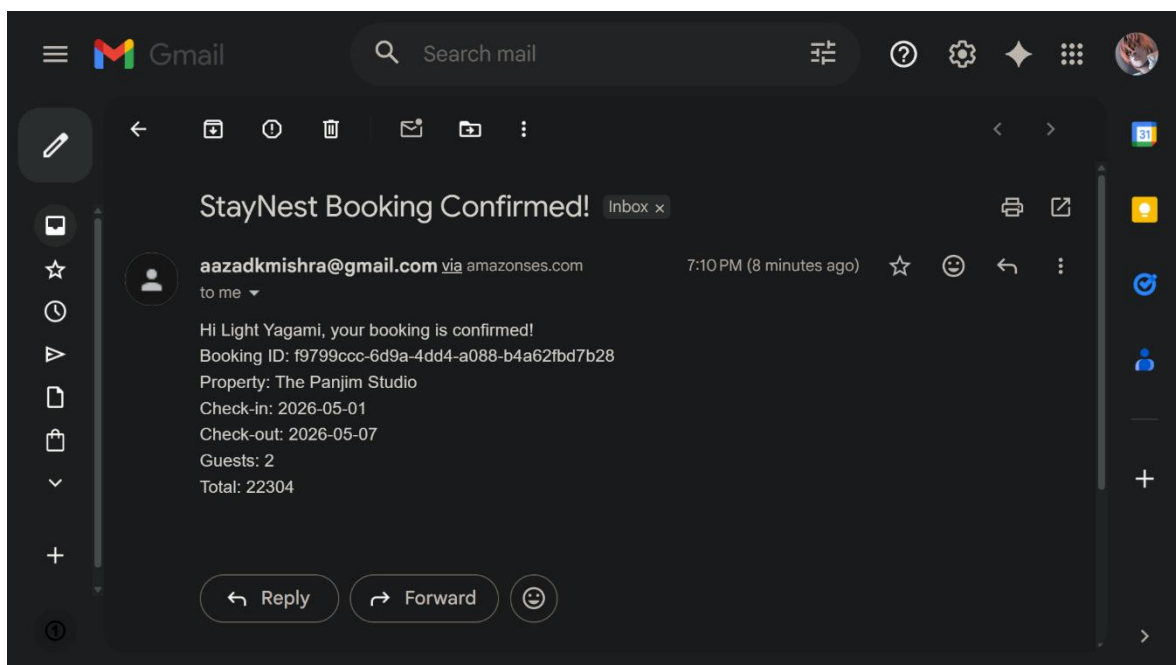
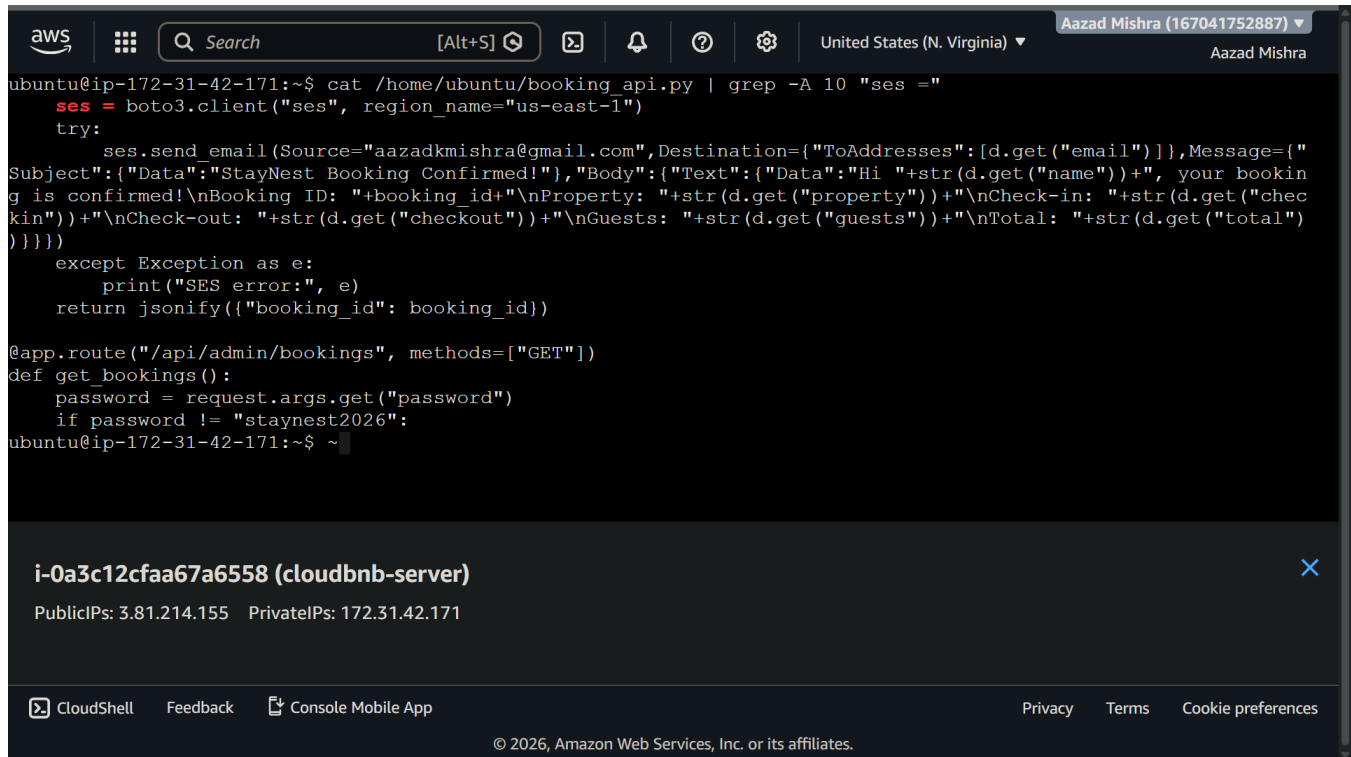


Figure 11: Customer Booking Confirmation Email



```

aws
Search [Alt+S]
United States (N. Virginia)
Aazad Mishra (167041752887)
Aazad Mishra

ubuntu@ip-172-31-42-171:~$ cat /home/ubuntu/booking_api.py | grep -A 10 "ses ="
    ses = boto3.client("ses", region_name="us-east-1")
    try:
        ses.send_email(Source="aazadkmishra@gmail.com", Destination={"ToAddresses": [d.get("email")]}, Message={
            "Subject": {"Data": "StayNest Booking Confirmed!"}, "Body": {"Text": {"Data": "Hi " + str(d.get("name")) + ", your booking is confirmed!\nBooking ID: " + booking_id + "\nProperty: " + str(d.get("property")) + "\nCheck-in: " + str(d.get("checkin")) + "\nCheck-out: " + str(d.get("checkout")) + "\nGuests: " + str(d.get("guests")) + "\nTotal: " + str(d.get("total"))}}}})
    except Exception as e:
        print("SES error:", e)
    return jsonify({"booking_id": booking_id})

@app.route("/api/admin/bookings", methods=["GET"])
def get_bookings():
    password = request.args.get("password")
    if password != "staynest2026":
ubuntu@ip-172-31-42-171:~$ ~

i-0a3c12cfaa67a6558 (cloudbnb-server)
PublicIPs: 3.81.214.155 PrivateIPs: 172.31.42.171

CloudShell Feedback Console Mobile App
Privacy Terms Cookie preferences
© 2026, Amazon Web Services, Inc. or its affiliates.

```

Figure 12: booking\_api.py SES integration snippet

### Flask SES Integration Snippet:

```

ses = boto3.client("ses", region_name="us-east-1")
ses.send_email(
    Source="aazadkmishra@gmail.com",
    Destination={"ToAddresses": [customer_email]},
    Message={
        "Subject": {"Data": "StayNest Booking Confirmed!"},
        "Body": {
            "Text": {
                "Data": "Booking confirmation details..."
            }
        }
    }
)

```

## 3.3 Admin Dashboard (Custom Feature Enhancement)

To enhance the usability and completeness of the system, a custom admin dashboard was developed and deployed. This dashboard provides a secure, real-time web interface for viewing all booking records stored in the RDS MySQL database without requiring direct terminal or database access.

### 3.3.1 Backend – Flask API Endpoint

A new route was added to the existing Flask booking API (/home/ubuntu/booking\_api.py). The endpoint /api/admin/bookings accepts a GET request with a password query parameter. If the password does not match, it returns a 401 Unauthorized response. If authenticated, it queries the RDS MySQL database and returns all booking records as a JSON array, with datetime fields serialised using .isoformat() for JavaScript compatibility.

```
@app.route("/api/admin/bookings", methods=["GET"]) def admin_bookings(): password = request.args.get("password") if password != "staynest2026": return jsonify({"error": "Unauthorized"}), 401 try: db = get_db() cursor = db.cursor(dictionary=True) cursor.execute("SELECT * FROM bookings ORDER BY created_at DESC") rows = cursor.fetchall() cursor.close(); db.close() for row in rows: for key, value in row.items(): if hasattr(value, 'isoformat'): row[key] = value.isoformat() return jsonify(rows) except Exception as e: return jsonify({"error": str(e)}), 500
```

### 3.3.2 Nginx Routing Configuration

By default, Nginx's try\_files directive attempts to serve requests as files using the exact URI. Visiting /admin would look for a file literally named admin rather than admin.html, resulting in a 404 error. A custom location block was added to /etc/nginx/sites-enabled/default to map the /admin route to the admin.html file:

```
location = /admin { try_files /admin.html =404; } location / { try_files $uri $uri/ =404; }
```

After editing the config, the Nginx configuration was validated with nginx -t and reloaded with systemctl reload nginx to apply changes without downtime.

### 3.3.3 Frontend – Admin Dashboard Page

A static admin.html file was created at /var/www/html/admin.html and is served directly by Nginx. It consists of two screens: a login screen that accepts the admin password, and the main dashboard that displays booking data in a table. The JavaScript fetch() API is used to call the Flask backend, and the response is rendered dynamically into the page without any page reload.

### 3.3.4 Dashboard Features

The admin dashboard includes the following features:

- Password-protected login screen — prevents unauthorised access to booking data
- Live booking table — displays all fields: Booking ID, Guest Name, Email, Property, Location, Check-in, Check-out, Guests, Total, Booked At
- Statistics panel — shows Total Bookings, Total Revenue (₹), and Total Guests aggregated from live data
- Manual refresh button — reloads booking data from RDS without a page reload
- Real-time updates — every new booking submitted through the website appears automatically on the next refresh

Access URL: <http://3.81.214.155/admin>

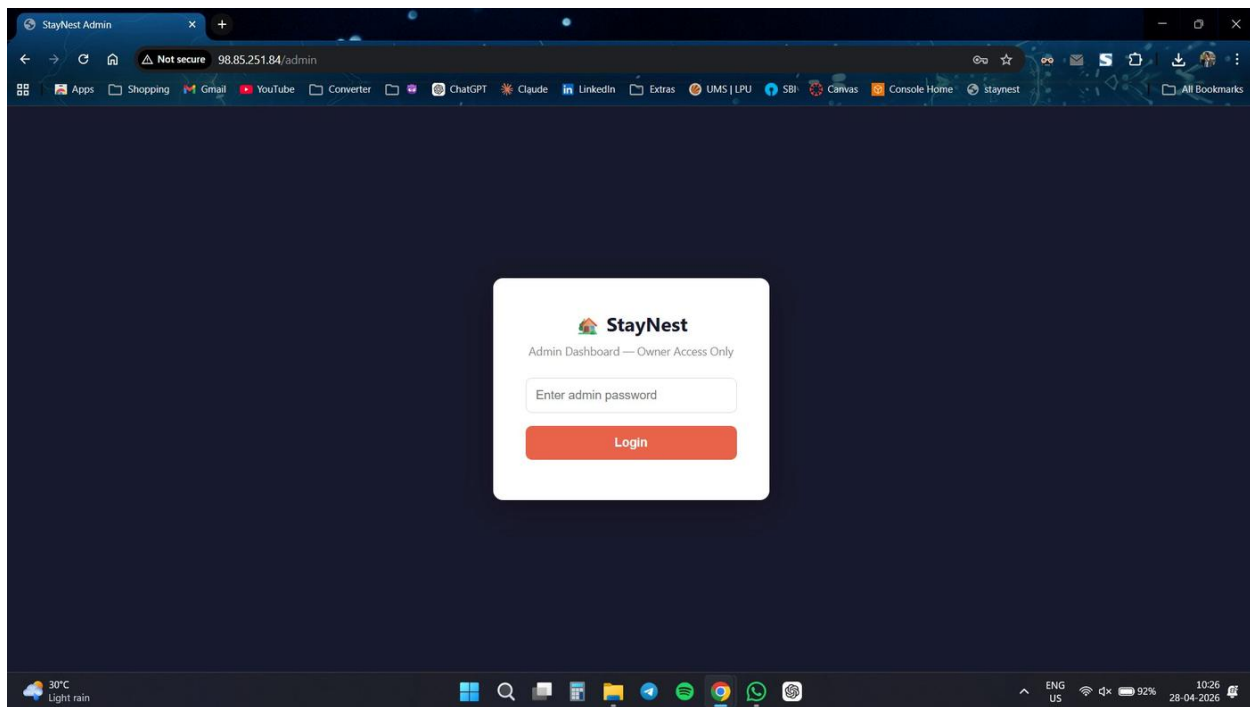


Figure 13: Admin dashboard login screen (<http://3.81.214.155/admin>)

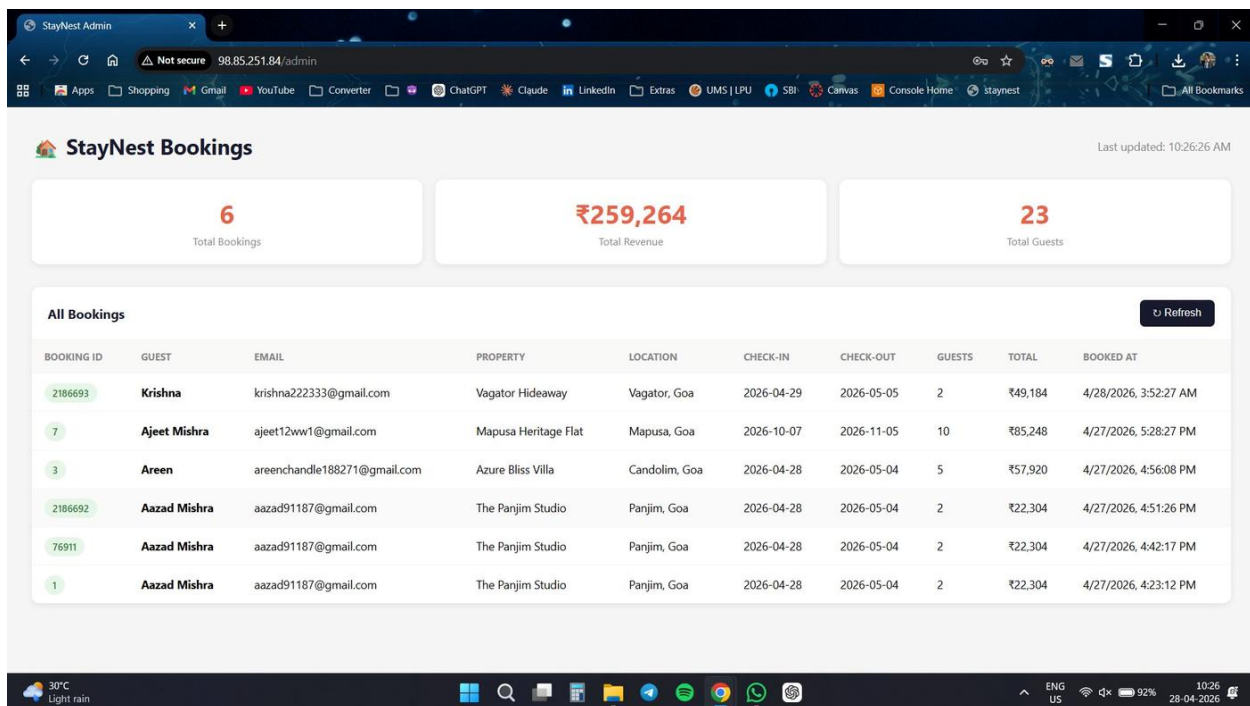


Figure 14: Admin dashboard showing 6 live bookings from RDS database



## 4. Architecture Overview

---

The following describes the complete request flow through the StayNest AWS architecture:

- User accesses the StayNest platform via the Application Load Balancer at `staynest-asg-1-1301223752.us-east-1.elb.amazonaws.com` (the primary endpoint) or the legacy direct IP `http://3.81.214.155/`
- Request is received by the Application Load Balancer (staynest-asg-1), which forwards traffic to a healthy EC2 instance in the Auto Scaling Group (staynest-asg). The target EC2 instance runs Nginx, which serves the HTML/CSS/JS frontend
- Property images are loaded from S3 via the CloudFront CDN (`d2unxbzahzt02h.cloudfront.net`), served from the nearest edge location
- When a user submits a booking form, the request is sent to the backend API on EC2
- The booking data is written to the RDS MySQL database (staynest-db)
- An SNS notification is triggered, sending booking details to the owner, while Amazon SES sends a confirmation email directly to the customer
- CloudWatch continuously monitors EC2 CPU utilisation and triggers SNS alerts if usage exceeds 80%
- All resources are secured within the default VPC, with IAM user staynest-dev holding read-only access for developer inspection
- The site owner can visit `http://3.81.214.155/admin` to view the password-protected admin dashboard, which fetches and displays all booking records live from RDS in a real-time table

## 5. Security Measures

---

Security was implemented across multiple layers of the architecture:

- IAM Least Privilege: The staynest-dev IAM user has only read-only access to S3, RDS, and CloudWatch — no write, delete, or administrative permissions.
- S3 Bucket Policy: Images are publicly readable via a bucket policy (`s3:GetObject` for all principals) rather than ACLs, following current AWS recommendations.
- VPC Network Isolation: All compute and database resources operate within a private VPC. Security groups restrict inbound traffic — only HTTP (port 80) is open on EC2, and RDS only accepts connections from the EC2 security group.
- CloudFront OAC: CloudFront uses Origin Access Control (OAC) to securely access the S3 bucket, providing an additional layer of access control over content delivery — CloudFront acts as the secure intermediary.
- No Hardcoded Credentials: All AWS credentials are managed through IAM roles and policies, not embedded in application code.

## 6. Performance & Scalability

Several architectural decisions directly improve performance and support future scalability:

### 6.1 Performance

- CloudFront CDN reduces image load latency by serving assets from edge locations globally rather than the S3 origin in us-east-1.
- Nginx is used as the web server due to its event-driven, non-blocking architecture — capable of handling significantly more concurrent connections than Apache on the same hardware.
- RDS managed database removes the performance overhead of a self-managed MySQL server on EC2.

### 6.2 Scalability

- The architecture has been fully scaled: an Application Load Balancer (staynest-asg-1) and Auto Scaling Group (staynest-asg) were implemented and are live. The ALB distributes incoming traffic across EC2 instances, and the ASG automatically scales from a minimum of 1 to a maximum of 3 t3.micro instances based on CPU utilisation — triggered by the existing staynest-cpu-alarm at the 80% threshold. The new primary endpoint is: staynest-asg-1-1301223752.us-east-1.elb.amazonaws.com

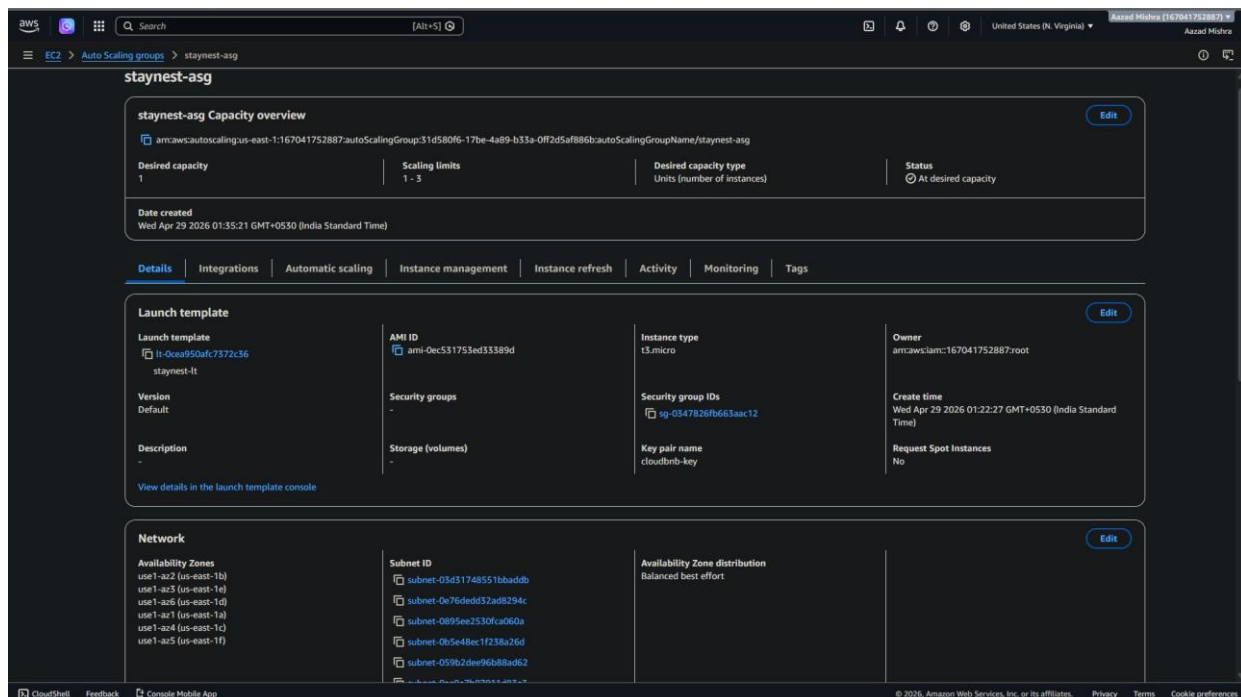


Figure 15: Auto Scaling Group (staynest-asg) – Desired capacity 1, Scaling limits 1–3, Status: At desired capacity

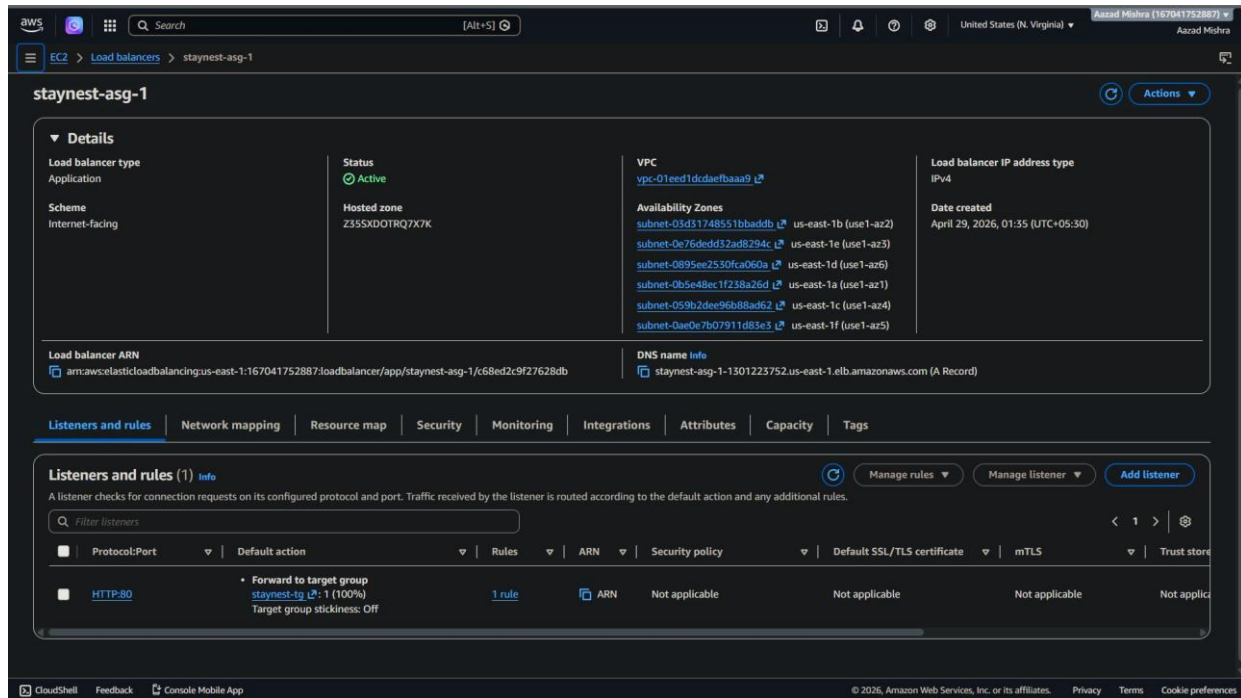


Figure 16: Application Load Balancer (staynest-asg-1) – Active, Internet-facing, HTTP:80 forwarding to staynest-tg across 6 Availability Zones

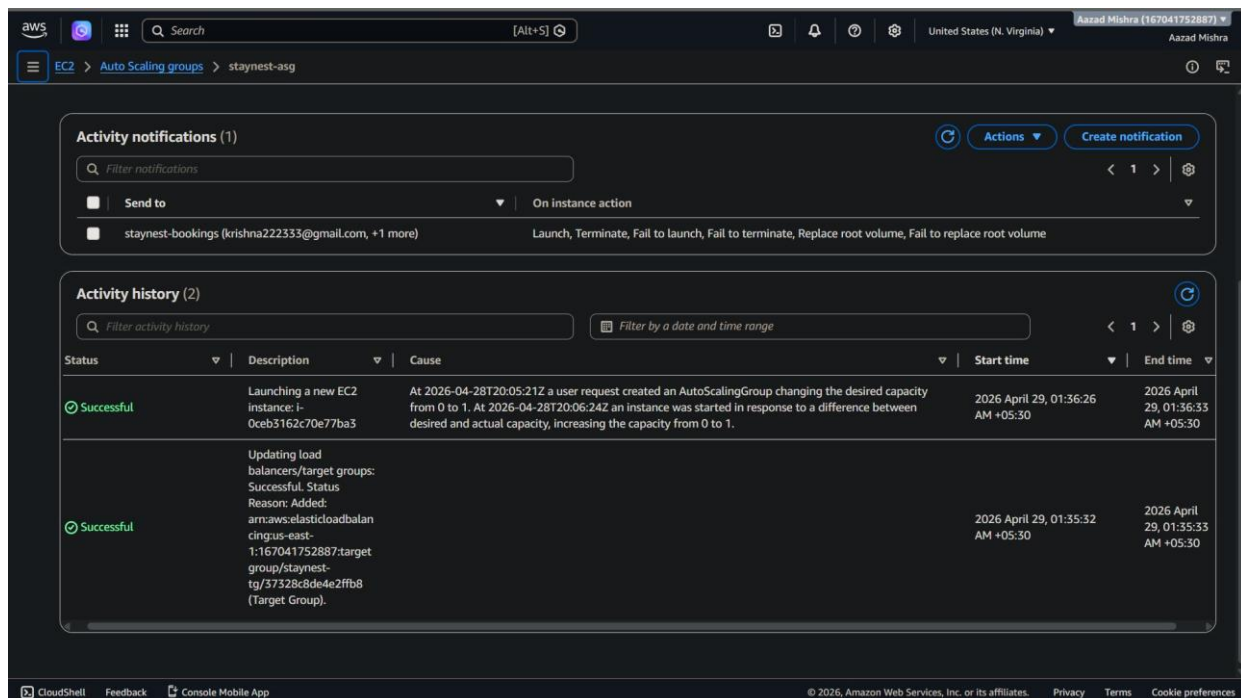


Figure 17: ASG Activity History – 2 successful events: EC2 instance launched and load balancer target group registered (April 29, 2026)

- S3 and CloudFront scale inherently — S3 handles unlimited objects, and CloudFront handles global traffic distribution without configuration changes.
- RDS supports read replicas for horizontal database scaling as booking volume grows.

## 7. Cost Management

---

The project was designed to minimise costs while meeting all functional requirements:

| Service    | Tier Used                                 | Estimated Cost |
|------------|-------------------------------------------|----------------|
| EC2        | t2.micro – Free Tier (750 hrs/month)      | \$0.00         |
| S3         | Free Tier (5GB storage, 20K GET requests) | \$0.00         |
| RDS        | db.t4g.micro – Free Tier (750 hrs/month)  | \$0.00         |
| CloudFront | Free Tier (1TB data transfer out/month)   | \$0.00         |
| CloudWatch | Free Tier (10 metrics, 10 alarms)         | \$0.00         |
| SNS        | Free Tier (1M requests/month)             | \$0.00         |
| IAM / VPC  | Always free                               | \$0.00         |

All services used fall within the AWS Free Tier limits for a new account. WAF was explicitly disabled on CloudFront to avoid the \$14/month charge. This architecture demonstrates that a fully functional, production-grade web application can be deployed on AWS at zero cost during development and testing phases.

## 8. Monitoring & Observability

---

The architecture includes proactive monitoring through CloudWatch and SNS:

- **EC2 CPU Monitoring:** The staynest-cpu-alarm CloudWatch alarm tracks CPU utilisation every 5 minutes. If the threshold of 80% is exceeded, an SNS alert is triggered.
- **SNS Email Alerts:** The owner receives immediate email notifications for infrastructure alarms and booking events, enabling rapid response to issues.
- **SES Customer Emails:** Customers receive automated booking confirmation emails upon successful booking submission.
- **RDS Monitoring:** RDS provides built-in performance insights and CloudWatch metrics for database connections, CPU, and storage — visible in the AWS console.
- **CloudFront Analytics:** CloudFront provides detailed request logs and cache hit/miss ratios, enabling optimisation of content delivery performance.

## 9. Conclusion

The StayNest project successfully demonstrates the design and deployment of a complete, multi-service AWS architecture for a real-world web application. All five required basic services (EC2, S3, IAM, VPC, RDS) were implemented and configured, along with three advanced services (CloudFront, CloudWatch, SNS) — exceeding the minimum requirement of two advanced services.

The architecture adheres to AWS best practices including least-privilege IAM policies, bucket policy-based S3 access control, VPC network isolation, and CDN-based content delivery. The application is live, functional, and fully independent of any third-party deployments.

This project provided hands-on experience in building a scalable, secure, and cost-effective cloud infrastructure — directly applicable to real-world AWS deployments and aligned with the learning objectives of INT 330. The project was further enhanced with the implementation of an Application Load Balancer (staynest-asg-1) and Auto Scaling Group (staynest-asg), bringing the architecture to a genuinely production-grade level with automatic traffic distribution and elastic scaling. The system was further enhanced with Amazon SES integration, enabling real-time customer booking confirmation emails, improving both usability and realism of the application.

## 10. Challenges Faced & Debugging

### Challenge 1: Nginx 404 on /admin Route

Visiting `http://3.81.214.155/admin` returned a 404 Not Found error even though the `admin.html` file existed at `/var/www/html/admin.html`. The root cause was Nginx's default `try_files` behaviour, which looked for a file literally named `admin` (without the `.html` extension) rather than `admin.html`. This was resolved by adding a dedicated `location = /admin` block to the Nginx configuration that explicitly maps the route to the correct file. The fix was applied using `sudo sed` and verified with `nginx -t` before reloading.

### Challenge 2: Database Column Name Mismatch (booked\_at vs created\_at)

The admin dashboard was loading but showing 0 bookings despite the API returning data successfully. The Flask query used `ORDER BY booked_at DESC`, but the actual column name in the RDS MySQL bookings table was `created_at`. This mismatch caused the query to fail silently. The fix involved updating the SQL query in `booking_api.py` using `sed` to replace `booked_at` with `created_at`, followed by a service restart. The same mismatch existed in the JavaScript `renderTable` function, where `b.booked_at` was referenced instead of `b.created_at`, requiring a second `sed` fix on `admin.html`.

### Challenge 3: JavaScript TypeError on Numeric Booking IDs

Even after fixing the column mismatch, the dashboard still displayed 0 bookings. Inspecting the browser console revealed a JavaScript `TypeError: b.id.substring is not a function`. The `renderTable` function called `.substring(0, 8)` on `b.id` to create a short display ID, but the `id` column in the database stored numeric integers (e.g. 1, 76911) rather than UUID strings. Calling `.substring()` on a number throws an error in JavaScript, which caused the entire render loop to fail silently. The fix was to convert the ID to a string first: `String(b.id).substring(0, 8)`, applied via `sed` on `admin.html`.

### Challenge 4: Debugging Methodology

Throughout debugging, two tools were essential. First, `curl` was used from the EC2 terminal to call the Flask API directly (`curl "http://localhost:5000/api/admin/bookings?password=staynest2026"`), confirming that the backend was returning valid JSON even when the dashboard showed nothing. This isolated the problem to the frontend. Second, the browser's Developer Tools (Network tab and Console) were used to inspect the actual HTTP response and identify the JavaScript error. This combination of server-side and client-side debugging confirmed the root cause at each stage.

## Updated Conclusion Addendum

The project was further enhanced by implementing a custom admin dashboard, providing real-time visibility into booking data. Additionally, an Application Load Balancer (staynest-asg-1) and Auto Scaling Group (staynest-asg) were successfully deployed, enabling the architecture to automatically scale from 1 to 3 EC2 instances in response to CPU load — completing a production-grade, fully elastic AWS deployment. Resolving real-world deployment and debugging challenges significantly strengthened practical cloud engineering skills.

## Appendix – Service Summary Table

---

| Service            | Resource Name                    | Purpose                                                                    | Status    |
|--------------------|----------------------------------|----------------------------------------------------------------------------|-----------|
| EC2                | cloudbnb-server                  | Nginx web server hosting frontend                                          | Running   |
| S3                 | cloudbnb-aazad-2026              | Property image storage                                                     | Active    |
| IAM                | staynest-dev                     | Least privilege developer user                                             | Active    |
| VPC                | vpc-01eed1dcdaefbaaa9            | Network isolation (172.31.0.0/16)                                          | Active    |
| RDS                | staynest-db                      | MySQL booking database                                                     | Available |
| CloudFront         | staynest-cdn                     | CDN for S3 image delivery                                                  | Deployed  |
| CloudWatch         | staynest-cpu-alarm               | CPU utilisation monitoring (>80%)                                          | Active    |
| SNS                | Email Notifications              | Booking confirmations + alarm alerts                                       | Active    |
| SES                | Verified Emails                  | Customer booking confirmation emails                                       | Active    |
| Admin Dashboard    | admin.html + /api/admin/bookings | Password-protected live bookings view                                      | Active    |
| ALB                | staynest-asg-1                   | Internet-facing load balancer distributing traffic across ASG instances    | Active    |
| Auto Scaling Group | staynest-asg                     | Auto-scales EC2 instances (min 1, desired 1, max 3) triggered by CPU alarm | Active    |